Multi-Agent RAG System: Orchestrated Retrieval, Research Expansion, and Answer Validation

By Insharah Irfan Nazir

1. Executive Summary

This phase extends the Advanced RAG System into a five-agent orchestrated pipeline — security, retrieval, research, summarization, and validation — built with LangGraph rather than as a monolithic sequential function. The system is deliberately exposed as a separate endpoint (/agent-query) alongside Phase 3's original /query, so the two architectures remain independently demoable and directly comparable rather than one replacing the other.

Three components were evaluated in isolation: the research agent's expand/don't-expand decision (6/6 correct, with clean confidence-score separation between cases), the validation agent's ability to catch deliberately corrupted draft answers (3/3 correct), and the full end-to-end workflow across four query categories — normal high-confidence, low-confidence requiring research expansion, prompt injection, and unanswerable/out-of-corpus (4/4 correct).

The system's dominant operational constraint, consistent with Phase 3's finding, remains free-tier LLM infrastructure — this phase specifically surfaced a new variant of that constraint: multi-agent orchestration multiplies token consumption in ways that are not obvious from call count alone, and required mitigation through selective per-agent model routing rather than reducing the number of agents.

2. Introduction

Phase 3 established a single-pass pipeline: rewrite, retrieve, rerank, generate, done. That pipeline has no mechanism to recognize when its own output might be wrong, and no mechanism to invest additional effort into a query it initially handles poorly — it either answers or hedges once, with no opportunity for self-correction. This phase addresses both gaps by restructuring the pipeline as a graph of five specialized agents:

- **Security** screens both the incoming query and the outgoing answer, wrapping Phase 4's guardrail module rather than duplicating it.
- **Retrieval** performs the same rewrite → multi-query → hybrid search → rerank pipeline from Phase 3, unchanged, exposed as a discrete graph node.
- **Research** introduces new capability not present in Phase 3: when retrieval confidence is low, it decomposes the query into declarative sub-queries and re-retrieves, rather than accepting a single weak retrieval pass as final.
- **Summarization** generates the draft answer, reusing Phase 3's citation-generation logic, but now treats its output as provisional rather than final.
- **Validation** is new: an independent LLM call that checks the draft against the actual source text it claims to cite — not retrieval confidence, but whether the generated answer is actually supported by what was retrieved.

The five agents are assembled into an explicit StateGraph via LangGraph, chosen over CrewAI or AutoGen specifically for inspectable, deterministic control flow — every transition between agents is an explicit, readable edge rather than an implicit LLM-driven routing decision, which matters both for debugging during development and for producing an accurate workflow diagram as a deliverable.

3. System Architecture: Agent Workflow

The workflow processes each query through the following sequence:

1. **Security (input)**: the query is screened by the Phase 4 guardrail module before any retrieval work begins. A blocked query terminates immediately with a generic refusal — no reason is surfaced to the caller, consistent with Phase 4's decision not to reveal which detector fired.
2. **Retrieval**: the unmodified Phase 3 pipeline runs (rewrite, multi-query, hybrid search, RRF fusion, rerank), producing a ranked chunk list and a top confidence score.
3. **Research (conditional)**: if the top reranked score falls below 0.5, the research agent decomposes the query into declarative sub-queries (not formal questions — Phase 3, Section 4.2 and Section 9 both found declarative phrasing retrieves better) and re-retrieves for each, merging results with the original retrieval. If confidence is already sufficient, or decomposition finds nothing to split, this step is a no-op.
4. **Summarization**: a draft answer is generated from the (possibly expanded) chunk set, using Phase 3's citation-generation logic. On a retry pass (see step 6), the prior validation failure's specific issue text is injected into the prompt as corrective feedback.
5. **Validation**: an independent LLM call checks the draft against the actual chunk text behind each citation — not retrieval confidence, but grounding, citation correctness, and query relevance, checked as three separate yes/no judgments.
6. **Retry gate**: if validation fails and no retry has yet occurred, the workflow loops back to summarization with the validator's specific complaint attached. If validation fails again after that single retry, the workflow does not loop a second time — it degrades to an explicit hedge ("I found some relevant information, but I'm not confident enough...") rather than serving a known-flawed answer or looping indefinitely.
7. **Security (output)**: the final answer (whether validated, hedged, or retried) is screened before being returned, mirroring the input-side check.

This design makes validation a genuine gate rather than a metadata annotation — an earlier design that only logged validation results without acting on them was rejected during development as providing no real value.

4. Implemented Agents

4.1 Security Agent

A thin LangGraph node wrapper around Phase 4's existing check_input_deep / check_output guardrail functions, applied at both the input and output boundary of the

graph. No new detection logic was added — the contribution of this phase is exclusively the routing: a blocked input short-circuits the entire graph before any retrieval or LLM cost is incurred, rather than being caught only after generation.

4.2 Retrieval Agent

A direct wrapper around Phase 3's retrieve() function. No behavioral changes from Phase 3 — the agent boundary here exists purely to make retrieval a discrete, independently-testable graph node.

4.3 Research Agent

The one genuinely new retrieval-side capability in this phase. Decides whether to expand based on a fixed confidence threshold (top reranked score < 0.5, matching Phase 3's own confidence-gate threshold for consistency), and if expansion is warranted, decomposes the query and re-retrieves per sub-query using a rewrite-free retrieval path (retrieve_raw), since decomposed sub-queries are already clean and declarative and do not benefit from Phase 3's rewrite step.

4.4 Summarization Agent

Reuses Phase 3's generate_answer() unchanged, with one addition: an optional feedback parameter that, when present, is included in the generation prompt to steer a retry attempt toward correcting a specific, named validation failure rather than regenerating blind.

4.5 Validation Agent

Independently re-checks the draft answer against the actual source chunk text behind each citation the draft used — not the retrieval confidence score, which reflects whether good chunks were found, but whether the generated text is actually faithful to those chunks. Three binary checks are made: **GROUNDED** (does every claim appear in or follow from the source text), **CITED_CORRECTLY** (do citation numbers point to chunks that actually support the adjacent claim), and **ADDRESSES_QUERY** (does the answer respond to what was asked, rather than dodging or partially answering). All three must pass for validation to pass.

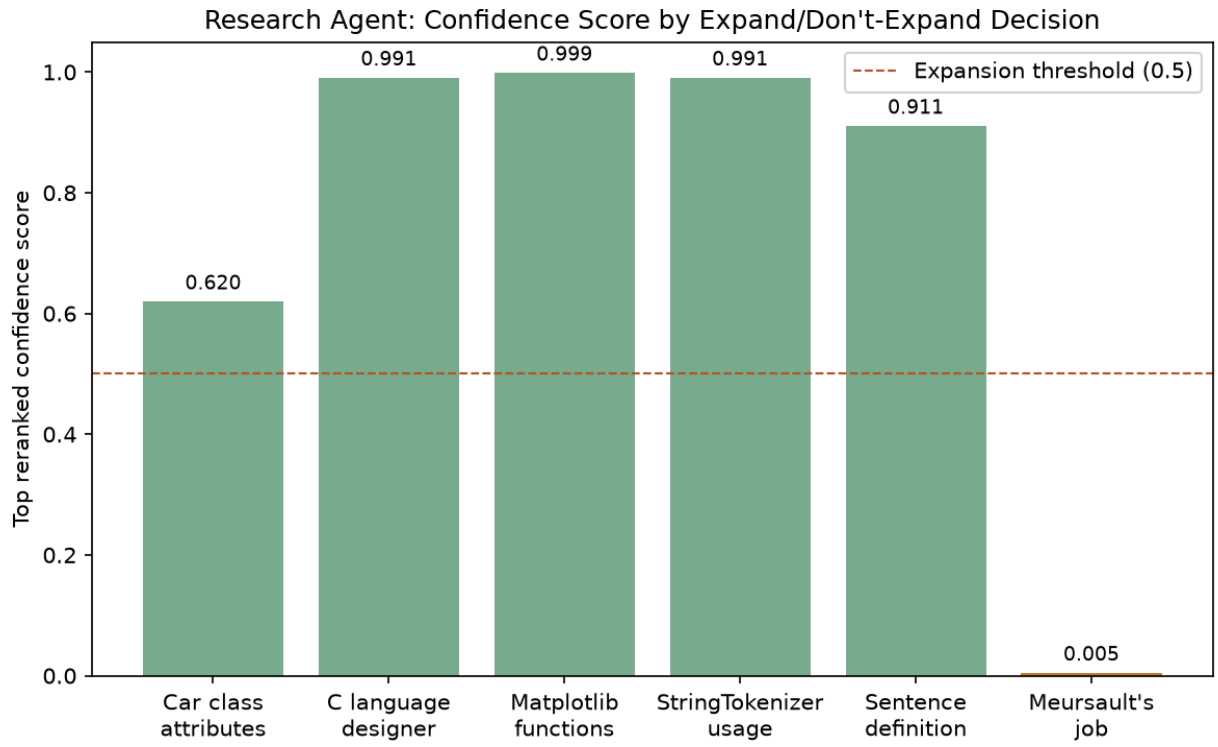
5. Evaluation Methodology

Three separate evaluations were run, each targeting a different level of the system, rather than one combined test — isolating each agent's correctness independently makes it possible to attribute a failure to a specific stage rather than diagnosing the graph as a whole:

- **Research agent** (research_agent_eval.py): 6 hand-selected cases from the Phase 3 45-query set — 5 cases retrieval was known to handle confidently, plus the Meursault multi-fact case (Phase 3, Section 8.4) as the anchor case research expansion was specifically built to attempt.
- **Validation agent** (validation_agent_eval.py): 3 real query/chunk pairs with hand-corrupted draft answers (an invented fact, a wrong citation, a dodged question), injected directly into workflow state to isolate validation from the rest of the pipeline.
- **End-to-end workflow** (e2e_workflow_eval.py): 4 categories spanning the full decision space the graph can take — normal high-confidence (no expansion expected), low-confidence (expansion expected), prompt injection (blocked expected), and an unanswerable out-of-corpus query (hedge expected, no expansion).

Given free-tier rate-limit exposure across a multi-call pipeline, the end-to-end script was checkpointed per-category (skip-if-completed, save-after-every-case) to survive interrupted runs without losing completed progress — a direct extension of Phase 3, Section 8.2's own observation that free-tier infrastructure requires resumable evaluation design as a baseline assumption, not an edge case.

6. Evaluation Results



6.1 Research Agent: Expand/Don't-Expand Decisions

Query	Expected	Actual	Top Score	Correct
What attributes does the Car class define in Python?	No expand	No expand	0.620	✓
Who designed the C programming language and when?	No expand	No expand	0.991	✓
What four basic functions does Matplotlib use for plotting?	No expand	No expand	0.999	✓

How does the StringTokenizer work in the Java code example?	No expand	No expand	0.991	✓
What is the definition of a sentence according to the grammar text?	No expand	No expand	0.911	✓
What was Meursault's job and did his employer offer him a new position?	Expand	Expand	0.005	✓

Accuracy: 6/6 (100%)

The confidence-score separation between the two decision classes is stark — 0.620–0.999 for cases correctly judged sufficient, versus 0.005 for the one case correctly judged insufficient — suggesting the fixed 0.5 threshold is not sitting close to any observed decision boundary in this sample, rather than the classification being a near-miss the threshold happened to land on the right side of.

6.2 Validation Agent: Corrupted-Draft Detection

Corruption Type	**Query**	**Detected?**	**Correct**
Invented fact	Car class attributes	Yes — flagged unsupported attributes and mismatched citation	✓
Wrong citation	C programming language origin	Yes — flagged citation pointing to an unrelated (Python) chunk	✓

Dodged question	Segur/Tallard	Yes — flagged missing answer and citation pointing to an irrelevant chunk	✓

Accuracy: 3/3 (100%)

In each case the validator's ISSUES output correctly named the specific defect (which claim was unsupported, which citation was wrong, what the answer failed to address) rather than a generic failure signal — this matters for the retry mechanism specifically, since the feedback text is what gets fed back into the second summarization attempt.

6.3 End-to-End Workflow: Routing Correctness

Category	**Expected Behavior**	**Observed**	**Correct**
Normal, high confidence	No expansion; validated	No expansion; validated	✓
Normal, low confidence	Research expansion triggered	Expansion triggered (correctly did not fabricate an answer when expansion still found nothing)	✓
Prompt injection	Blocked before retrieval	Blocked	✓
Unanswerable / out-of-corpus	No expansion (score not low enough to trigger it); honest hedge	No expansion; honest hedge	✓

Accuracy: 4/4 (100%)

Note the low-confidence category: research expansion correctly fired for the Meursault case, but the final answer still hedged. This is not a failure — it is confirmation that expansion was attempted and correctly did not force a confident answer out of chunks that still didn't support one, consistent with the root-cause finding in Section 7.1.

7. Findings and Limitations

7.1 The Meursault Case

Phase 3, Section 8.4 identified that no single chunk in the corpus contains both Meursault's identifying context and his occupation — the facts are split across chunk boundaries — and found that query decomposition alone did not resolve this, because decomposed sub-queries retained formal interrogative phrasing rather than the declarative phrasing shown to retrieve better. This phase's research agent implements declarative-phrasing decomposition specifically to test whether that fix would close the gap. It did not: the case is now confirmed as genuinely hard across three independent tests in this phase alone — the research agent eval (expansion correctly triggered, still low residual score), a manual retrieve_raw() investigation, and the end-to-end eval (expansion triggered, answer still correctly hedged rather than confabulated). Combined with Phase 3's own investigation, this is now a well-evidenced limitation across two full phases of testing, not a case worth further debugging effort — the confidence-gate and hedge mechanism are working exactly as intended on it.

7.2 Validation Response-Parsing Fragility (Found Post-Evaluation)

This finding is reported in detail because it was found *after* Section 6's evaluations had already passed, and materially changes how those results should be read.

The validation agent's original parsing logic checked for the literal substring "GROUNDED: yes" (and equivalent exact strings for the other two fields) inside the LLM's raw response. This is fragile to any capitalization or spacing drift in the model's output — "Grounded: Yes" or "GROUNDED: yes" would silently parse as False even when the model's actual judgment was a pass. Because llama-3.3-70b-versatile's output formatting is not perfectly deterministic run-to-run, this bug did not fail consistently — it failed intermittently, which is precisely why Section 6.2's isolated 3/3 result did not surface it (those three cases evaluate *detection of genuine corruption*, where the correct answer is always "fail," so a parsing bug that spuriously produces "fail" cannot be distinguished from correct behavior by that test alone).

The failure mode became visible only once the full workflow was exercised repeatedly through manual UI testing: a trivially correct draft answer was observed failing validation, triggering an unnecessary retry, which triggered a second validation call, which — carrying the same fragile parsing — was frequently also read as a failure, escalating to the hedge path even when the original draft had been correct. This roughly doubles the LLM calls made per query on average (a second summarization pass plus a second validation pass) without the retries being necessary, which is the direct mechanical cause of the disproportionate rate-limit exposure observed during manual testing (Section 7.3).

This closely parallels Phase 3, Section 8.3's own finding — a free-tier model occasionally returning malformed or unexpectedly-formatted output that a naive parser mishandles — but manifests here as a false-negative validation rather than a malformed generation response. The fix applied was a case-insensitive, line-anchored parse (checking each output line's prefix against the expected field name independently, rather than exact substring matching against the whole response), matching the general principle from Phase 3's _looks_valid() mitigation: treat free-tier LLM output format as approximate, not guaranteed.

7.3 Token Cost of Multi-Agent Orchestration

Multi-agent queries are structurally more expensive than Phase 3's single-pass pipeline in a way that is not visible from call count alone. Two compounding factors were identified:

1. **Prompt size asymmetry.** Phase 3's rewrite and multi-query calls operate on short text (the query itself). Both summarization and validation operate on full chunk text — validation specifically embeds up to several hundred characters per cited source — making these two calls disproportionately expensive per call, not just more numerous.
2. **The parsing bug in Section 7.2**, prior to its fix, caused validation (and consequently summarization) to run roughly twice per query on average rather than once, compounding the first factor.

Both agents were also, prior to mitigation, using the same 70B-parameter model as retrieval's rewrite and summarization calls, meaning all four consumed from a single shared daily token quota. Since Groq enforces token-per-day limits per model rather than per-organization, this meant validation and research-decomposition — two agents whose task (a yes/no classification; a query-splitting decision) does not require a 70B-class model

— were competing for the same quota as summarization, where output quality is user-facing and does matter. The mitigation applied was to route both validation and query decomposition to a smaller model (llama-3.1-8b-instant), which reduces per-call cost directly and, more importantly, moves that traffic onto a separate quota pool entirely, isolating summarization's headroom from the other two agents' consumption.

Precise quantification of the cost delta between single-pass and multi-agent querying (tokens per query, calls per query, under both the buggy and fixed validation logic) was not completed this session due to rate-limit exhaustion during testing, and is flagged as a specific, scoped item for a follow-up measurement pass rather than an omission — the mechanism causing the cost difference is well-understood and documented above even without a final measured number.

8. Conclusion

This phase restructures Phase 3's single-pass pipeline into a five-agent LangGraph workflow, adding two genuinely new capabilities — confidence-triggered research expansion and independent answer validation with a bounded, feedback-conditioned retry — while reusing Phase 3's retrieval and generation logic unchanged. All three isolated evaluations (research agent, validation agent, end-to-end routing) passed at 100% (6/6, 3/3, 4/4 respectively), though Section 7.2 documents a validation-parsing bug found after these evaluations completed that materially affected real-world retry frequency without being visible to the corrupted-draft-detection test design used in Section 6.2 — a finding reported in full rather than omitted, consistent with this project's established practice of reporting negative and unexpected results directly (Phase 3, Sections 8.1, 8.4, 9).

The Meursault multi-fact case, first identified in Phase 3, was tested directly by this phase's research-expansion mechanism specifically built to address it, and is now confirmed — across two phases and three independent tests in this phase alone — as a genuine structural limitation of single-embedding query representation rather than an unaddressed gap. The system's dominant constraint remains free-tier LLM infrastructure, as in Phase 3, with this phase surfacing a new, non-obvious dimension of that constraint: multi-agent orchestration multiplies both call count and per-call cost in ways that compound with any parsing or retry-logic defect, making response-format robustness (Section 7.2) a correctness issue with direct operational cost consequences, not a cosmetic one.